

Project report in DAT255 – Deep Learning Engineering

Detecting Diseases in Chest X-ray Images Using Deep Learning

25.04.25

Ole Fossbakk Birketvedt

Ivar Heggøy Nesheim

Fredrik Lauritzen

I confirm that the work is self-prepared and that references/source references to all sources used in the work are provided, cf. Regulation relating to academic studies and examinations at the Western Norway University of Applied Sciences (HVL), §10.

Content

Problem description.....	3
Introduction and motivation.....	3
Why deep learning?.....	3
Existing solutions.....	4
Aim of the project.....	4
Use of AI in the project.....	5
Code Availability and Application Deployment.....	5
Data.....	6
Dataset Description and Choice.....	6
Dataset pros and cons.....	7
Alternative datasets.....	8
Data preprocessing.....	8
Data augmentation.....	9
Model implementation.....	10
Model Architecture and Theoretical Motivation.....	10
Training procedure.....	12
Hyperparameters and optimization.....	12
Evaluation.....	14
Evaluation metrics.....	14
Evaluation baseline.....	15
Interpretation Techniques.....	16
Utilization of Model Architecture.....	16
Deployment.....	17
Conclusion.....	18
References.....	19

Problem description

Introduction and motivation

Chest radiography (X-ray) is one of the most common medical imaging procedures performed, critical for diagnosing various cardiopulmonary conditions [1]. However, accurate interpretation of these images demands specialized expertise and is time-consuming. This could lead to significant workload pressures and potential reporting delays, which can significantly impact the patient's care. This project employs deep learning techniques to develop a model capable of identifying multiple prevalent life-threatening diseases on chest X-rays. Utilizing the publically accessible CheXpert dataset provided by the Stanford Machine Learning Group [2], our model is trained and evaluated on a broad spectrum of conditions.

The CheXpert dataset comprises X-ray images annotated with 14 labels, each representing a distinct medical condition. The objective of this project is to develop a predictive model capable of identifying the presence of each of these conditions. The model outputs probabilistic estimates for each label while explicitly accounting for uncertainty in the data. To illustrate how the model could be utilized, it will be deployed via a user interface that enables users to upload chest X-ray images and receive diagnostic predictions.

Why deep learning?

Traditional machine learning methods, typically referring to supervised learning techniques such as linear or logistic regression, perform well when applied to tabular data with clearly defined, low-dimensional and interpretable features [3]. In such settings, engineers can manually select or engineer meaningful features, fit a model and tune hyperparameters using performance on a validation set. The final model is then evaluated on a validation set to estimate its generalization performance on unseen data. This workflow, however, relies on the assumption that the input features are both explicitly defined and human-interpretable.

In contrast, our input data consists of 320x320 pixel images, yielding over 100 000 features per sample. These features are neither intuitive nor easily interpretable by humans, and their sheer volume makes manual feature engineering impractical. Under these conditions, convolutional neural networks (CNN) offer a more suitable alternative. CNNs are particularly well-suited for high-dimensional image data, as they are capable of automatically learning hierarchical feature representation directly from raw pixels [4]. Early convolutional layers capture low-level patterns such as edges and textures, while deeper layers integrate these into more complex structures and potential pathological indicators, emulating the interpretive process of a trained radiologist. The ability to learn task-relevant features without manual intervention makes CNNs the optimal approach for medical image analysis tasks, which includes chest X-ray interpretation.

Existing solutions

The CheXpert dataset was released as part of a competition on automated chest X-ray interpretation organized by the Stanford Machine Learning Group [2]. A leaderboard available on the group's website showcases submitted models and their performance, providing valuable insight into existing state-of-the-art approaches. Among the top-performing entries were DeepAUC-v1 Ensemble and Hierarchical-Learning-V1 (Ensemble), each achieving an AUC of 0.930 [2]. Although neither of these solutions has made their source code publically available, both groups published academic papers detailing the methods they employed [5, 6]. Notably, both approaches reported their best results when using the DenseNet121 architecture.

Aim of the project

The aim of our project is not to surpass the performance of the top models from the competition. Those results were achieved through the use of substantial computational resources and deep domain expertise, which are beyond the scope of our student project. Instead, our aim is to achieve competitive results within realistic constraints, with a particular emphasis on:

- **Minimizing cost:** As a student-led project, it will operate without dedicated funding or access to premium tools. This means that the tools available to us are our own personal computers, as well as freely available online services.
- **Working with limited computing power:** In line with minimizing costs, we lack access to high-performance computing clusters. Model training will mainly be conducted on personal machines, which restricts our ability to conduct extensive hyperparameter tuning or experiment with large-scale model ensembles.
- **Exploring optimization techniques:** We will explore different optimization techniques such as learning rate schedulers, different optimizers, dropout rates, and other techniques from the course.
- **Deployment:** Unlike the competition entries, which focused exclusively on model performance, our project also includes a deployment component. We will develop a web-based user interface that allows users to upload chest X-ray images and receive predictions from the model. This feature is intended to highlight the practical utility of our approach in a real-world clinical support setting.

Use of AI in the project

In this project, we utilized AI tools to support both the development process and the writing of this report. During code development, we used Gemini 2.5 Pro for inspiration regarding implementation details and potential ways to improve model performance. While we considered the suggestions provided by the tool, we did not adopt them blindly. For example, some augmentations suggested would not make much sense for X-rays (e.g. cutout might obscure pathology), complex schedulers and models that were unsuitable for our constraints, and outdated or wrong information.

For writing the report, we used AI tools such as ChatGPT and Gemini to assist with language refinement. Our workflow involved first drafting each paragraph ourselves, then requesting feedback or improvements from one of the AI tools. We reviewed the suggested revisions and incorporated changes selectively, ensuring we fully understood the vocabulary and phrasing used. If a suggestion included words we did not understand, we replaced it with terminology that was consistent with our own understanding and communication style. We also made sure that the content of the paragraph remained the same before and after the revision, as we did not want the AI to add anything to our report. In this way, AI was used as a supportive tool to enhance clarity and structure, rather than as a substitute for our own writing.

Code Availability and Application Deployment

The source code used to train our model is publicly available in our GitHub repository:

<https://github.com/lvhene/dat255-project-group6/tree/main>

The repository includes a README.md file detailing the necessary prerequisites for running the code locally, as well as instructions for executing the Jupyter notebook on Google Colab.

The deployed web application can be accessed at the following link:

<https://huggingface.co/spaces/olefb/CheXpertLabeler>

The codebase for the deployed app is also publicly available within the corresponding Hugging Face repository:

<https://huggingface.co/spaces/olefb/CheXpertLabeler/tree/main>

Data

Dataset Description and Choice

The CheXpert dataset is a large-scale collection of chest radiographs compiled and labeled by the Stanford Machine Learning Group at Stanford University [2]. The dataset comprises 224,316 chest X-ray images from 65,420 patients, all conducted at Stanford Hospital between October 2002 and July 2017. Each radiograph is annotated with up to 14 clinical observations, derived from the corresponding radiology reports.

The training set in the CheXpert dataset is accompanied by three different CSV files, each reflecting a different labeling approach. The original training label file, `train.csv`, was generated using a rule-based natural language processing (NLP) system developed by the CheXpert team. This system extracted labels from radiologists' textual reports for each X-ray image, assigning values of 0 (negative), 1 (positive), -1 (uncertain) or blank (unmentioned) to each of the 14 conditions. This file served as the primary ground truth for most models trained on CheXpert prior to 2021 [13].

In 2021, an alternative labeling method was introduced using a BERT-based model called VisualCheXbert [13]. Unlike the original approach, VisualCheXbert incorporates both the radiology reports and the image data itself to improve alignment with radiologist interpretations. The resulting labels are considered by the authors to offer a more reliable representation of ground truth. For this reason, we chose to use the `train_VisualCheXbert.csv` file for our model training. This file provides binary labels, where a value of 1 indicates a positive finding, and a value of 0 represents either a negative or uncertain finding.

The dataset is pre-divided into training and validation subsets. The training set contains 223,414 images, while the validation set comprises 234 images. An additional test set of 668 images was used by the Stanford ML Group to evaluate and rank submissions in the CheXpert competition.

One limitation of working with the full CheXpert dataset, which contains 224,316 chest radiographs, is the considerable storage requirement it brings. The complete dataset occupies approximately 470 GB of disk space, far exceeding the storage capacity of most free cloud services and personal computing devices. Additionally, processing the full-resolution images during model training would demand significant computational resources. To remain within the practical constraints of this student project, namely minimizing use of paid services and high-performance hardware, we opted to use a downsampled version of the dataset made available by Ashery on Kaggle [7]. This version reduces the total dataset size to approximately 11.5 GB, making it feasible to use both on local machines and platforms such as Google Colab.

Although this version has not been independently verified by the original dataset creators, manual inspection of multiple samples confirms that the images and labels match those in the official CheXpert dataset. Furthermore, the total number of radiographs is consistent with the original release. However, it is important to note that this downsampled dataset does not include the separate test set used by the Stanford ML Group for ranking competition submissions. Despite this limitation, the reduced dataset provides a practical and accessible alternative that supports our goal of training and evaluating models within our available resources.

Dataset pros and cons

Using a downsampled version of the CheXpert dataset offers practical advantages, but also introduces certain trade-offs. Most notably, the reduced image resolution can result in the loss of fine-grained visual details that may be critical for detecting subtle pathologies such as small fractures. Consequently, this reduction in fidelity may lead to lower diagnostic performance, particularly for conditions that rely on high-resolution image features. The downsampled dataset may also limit our ability to take full advantage of more advanced models, as the reduced image resolution lacks the fine-grained details these models are designed to leverage. Nevertheless, as the primary advantage of using the downsampled dataset is accessibility, it allows us to conduct training and experimentation using our own machines and free platforms such as Google Colab, thereby staying within the resource constraints of this project.

The CheXpert dataset offers several benefits that make it highly suitable for deep learning tasks. With 224,316 chest radiographs from 65,420 patients, the dataset provides ample data for training complex models. It is multi-label in nature, allowing for simultaneous training across 14 common clinical observations. Furthermore, it is publicly available, promoting reproducibility in both research and educational settings. The dataset also includes a predefined split into training and validation sets, which simplifies experimental design and evaluation.

Another consideration is that some patients in the dataset have multiple radiographs. This could introduce bias, as repeated images from the same patient may contain similar findings, potentially inflating performance metrics if such cases appear in both the training and validation sets. While the dataset's predefined split helps avoid this issue, any effort to expand the validation set would require careful patient-level separation to prevent data leakage.

The validation set itself is relatively small, consisting of only 234 images. This limited size reduces the robustness of performance estimates and may not capture the full variability of the dataset. For example, there is not a single instance of the *fracture* condition in this validation set.

Finally, both the original and downsampled datasets include images with varying dimensions. As a result, we must resize all images during preprocessing. This resizing step, especially in combination with the lower resolution of the downsampled dataset, could further reduce model performance, particularly for detecting pathologies that depend on fine visual cues.

Alternative datasets

Several publicly available chest X-ray datasets exist for medical image analysis. Among the largest are the NIH ChestX-ray14 dataset [8], released by the National Institutes of Health, and the MIMIC-CXR dataset [9], developed through a collaboration between the Massachusetts Institute of Technology, Beth Israel Deaconess Medical Center, Stanford University, and Harvard Medical School [10]. In addition, smaller datasets such as the Lung Disease Classification dataset [11] and the 4-Types Lung Disease dataset [12], both hosted on Kaggle, offer more limited but focused resources.

For this project, we selected the CheXpert dataset due to several key advantages: its support for multi-label classification across 14 clinical observations, the availability of a downsampled version suitable for use within our computational constraints, and the presence of numerous existing solutions and research efforts based on this dataset. These factors made CheXpert particularly well-suited for our goals in both experimentation and evaluation.

Data preprocessing

The primary preprocessing steps for the chest X-ray images were as follows:

- **Loading:** Images were loaded using the Python Imaging Library (PIL), with file paths obtained directly from the provided CSV files.
- **Color Conversion:** Although chest X-rays are inherently grayscale, images were converted to RGB format by duplicating the single grayscale channel across three channels. This step ensures compatibility with pre-trained convolutional neural networks, which typically expect three-channel input.

- **Resizing:** All images were resized to a uniform resolution of 320×320 pixels. This resolution was chosen as a compromise between preserving anatomical detail and managing computational and memory constraints. Initially, a resolution of 224×224 pixels was used to match the standard ImageNet input size; however, we determined that adjusting up to 320×320 allowed us to retain more diagnostic information from the already downsampled dataset.
- **Tensor Conversion:** The images were converted to PyTorch tensors, with pixel values scaled to the range [0, 1].
- **Normalization:** To match the input expectations of models pre-trained on ImageNet, tensors were normalized using the standard ImageNet channel-wise mean [0.485, 0.456, 0.406] and standard deviation [0.229, 0.224, 0.225]. Normalization plays a crucial role in maintaining the consistency of input distributions during training and inference.

Data augmentation

To enhance data diversity and improve model generalization, a set of data augmentation techniques was applied exclusively to the training set:

- **Horizontal Flipping:** Images were randomly flipped along the horizontal axis. This transformation is considered medically plausible for chest X-rays, as anatomical asymmetry in many thoracic conditions is limited or diagnostically irrelevant in this context.
- **Rotation:** Images were randomly rotated by up to +/- 10 degrees. This augmentation simulates minor variations in patient positioning during radiograph acquisition, helping the model become more robust to real-world variability.
- **Color Jitter:** Images had their brightness and contrast randomly adjusted by up to +/- 20% each. This simulates variations in X-ray exposure levels and machine calibration, which should help make the model less sensitive to brightness and contrast levels.
- **Random Affine:** Images were shifted horizontally and vertically by up to 5%, simulating variations in the centering of the anatomy within the image frame. Additionally, a shear transformation of up to +/- 5 degrees simulated distortions that can occur due to misalignment between the patient and instruments.
- **Random Augment:** This is an automated augmentation technique which applies a sequence of randomly selected augmentation operations, added for its simple yet powerful way to further diversify the training data. With default settings, it adds two augmentations per input image.

Model implementation

Model Architecture and Theoretical Motivation

The final model architecture selected for this project is ResNet50, a convolutional neural network (CNN) well-established for image classification tasks. This choice was the result of extensive experimentation and was motivated by both theoretical considerations and empirical performance.

We evaluated several architectures, including DenseNet121, ResNet152, and ResNet50. Each was initialized with ImageNet pre-trained weights to benefit from transfer learning, which accelerates convergence and improves performance, especially valuable in medical datasets with class imbalance and label uncertainty.

While DenseNet121 has been a common choice among many top implementations on the CheXpert leaderboards, including in the CheXpert baseline paper, our experiments showed that it did not outperform ResNet50 on the downsampled dataset. Similarly, we trained ResNet152, a deeper variant of ResNet50, for 12 epochs across 18 hours of runtime on GPU. Despite its greater depth, it failed to surpass the performance of ResNet50, likely due to overfitting or underutilization of its depth given the lower resolution and smaller validation set size.

In addition to architecture changes, we experimented with more data augmentation, AdamW, learning rate scheduling, weight decay and dropout tuning. None of these changes consistently improved upon our ResNet50 baseline by a significant amount. Our final ResNet50 model achieved a mean AUC of 0.890, which was the best performance we obtained across all configurations.

ResNet50 thus provided the best balance between model complexity and practical performance. It was deep enough to learn meaningful representations, efficient enough to train within our resource constraints, and robust enough to generalize better than deeper or more aggressively regularized models.

A common observation with different settings was that the model quickly achieved its peak AUC, while training loss steadily decreased over following epochs. This indicates strong early learning and a subsequent tendency to overfit to the training data. Experimenting with various implementations increased the complexity of both the architecture and the codebase. This was useful, however, as it let us learn how to connect new parts to the foundation.

In conclusion, while DenseNet121 and ResNet152 are both theoretically strong architectures and widely used in the field, ResNet50 emerged as the most effective choice in our experimental setup. It demonstrated strong empirical performance while being computationally tractable, making it the optimal backbone for our CheXpert classification pipeline. Our model structure in the end was the following:

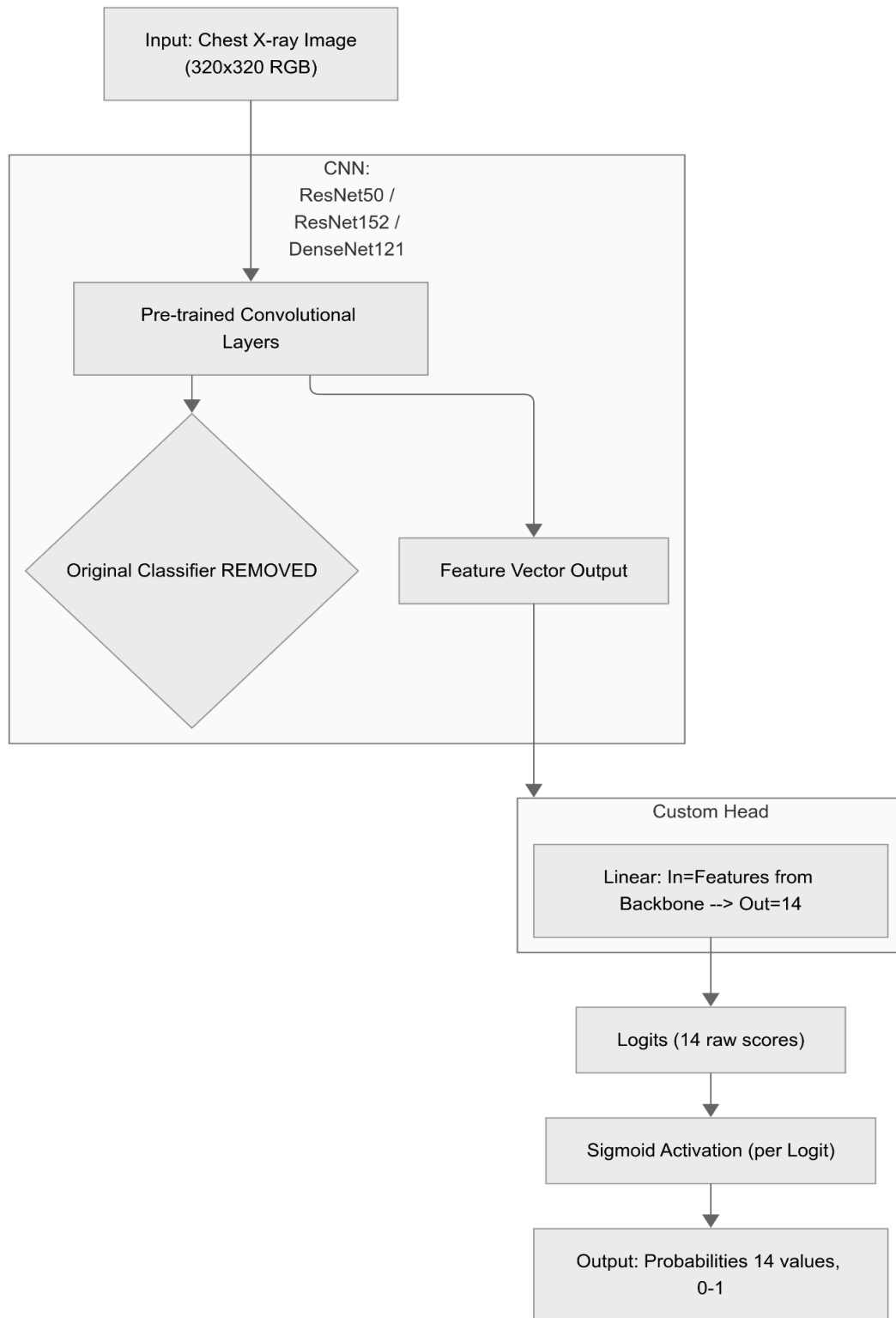


Figure 1: High-level architecture diagram showing the input image flow through the pre-trained backbone (with original head removed) and the new classification head.

Training procedure

PyTorch and TensorFlow (used together with the high-level API Keras) are the main frameworks for deep learning. They have similar performance and mostly the same syntax, but vary in the libraries available. Our choice fell on PyTorch 2.6.0 for this project, due to its more Pythonic syntax, ease of importing model weights and libraries like Grad-CAM.

The training code was written in a Jupyter notebook file which could be employed both in a locally installed PyTorch environment and on Google Colab with few changes required. With the help of AI, support was also added for Colab's TPU runtimes, but we ultimately found GPU runtimes more reliable.

The model state, optimizer state, scheduler state, current epoch, best validation AUC, and metric history were saved after each epoch that resulted in an improvement in validation AUC. This allowed for resuming training and ensured the best-performing model was retained.

We primarily used ResNet50 as our backbone architecture with a custom classification head. This model is both expressive and efficient, allowing us to exploit transfer learning via pre-trained ImageNet weights. The training and inference were performed on GPU, and performance per epoch was measured to be manageable within our constraints (~40 minutes per epoch on a laptop RTX 3070, and ~15 minutes per epoch on Colab's A100 GPU). We monitored training time and GPU memory consumption, which remained reasonable throughout, supporting the feasibility of deploying this model on standard cloud or high-end consumer hardware.

Finally, we monitored and plotted the evaluation metrics during and after training, and made adjustments to structure and hyperparameters based on these.

Hyperparameters and optimization

The Adam optimizer [14] was initially chosen for its proven performance in many deep learning tasks. Default parameters were used for the optimizer. Later, in an effort to mitigate overfitting and improve results, the AdamW optimizer was added with weight decay (L2 regularization). Weight decay values between $1e-5$ and $1e-4$ were experimented with. Weight decay adds a penalty term to the loss function proportional to the squared magnitude of the model's weights [15]. This encourages the optimizer to find solutions with smaller weights, which reduces model complexity and improves its ability to generalize to unseen data. The AdamW optimizer decouples the weight decay step from the gradient-based update and applies the penalty step directly to the weights. However, despite this theoretical advantage, no significant difference was observed between the two choices of optimizer, likely due to the limitations of the dataset used.

The batch size was a crucial number to adjust depending on the training environment (local or on Colab with various GPU/TPU runtimes) and which backbone model was being used. Larger batch sizes generally lead to faster training per epoch and potentially more stable gradients, but require more memory. For ResNet50 on the local machine, a batch size of 16 proved the most efficient and stable, while high-memory GPUs on Colab could take 64 or 128 images per batch.

For the loss function, PyTorch's *BCEWithLogitsLoss* was used. This loss function is suitable for multi-label classification. It combines a Sigmoid activation layer with the Binary Cross-Entropy loss, making it numerically stable. It computes the loss independently for each of the 14 labels against their respective target, and then averages them.

The learning rate (LR) was initialized at $1e-4$, a common starting point for fine-tuning with the Adam(W) optimizer. A scheduler was implemented with PyTorch's *ReduceLROnPlateau*, which monitored the mean validation AUC and reduced the LR by 0.1 if the AUC did not improve for 3 consecutive epochs. Additionally, a warmup scheduler was attempted in order to stabilize initial learning. In theory, both of these strategies help fine-tune the model when performance plateaus. However, there was no significant gain seen from scheduling.

Early Stopping was implemented to prevent overfitting and save training time. Training was halted if the mean validation AUC did not improve for `EARLY_STOPPING_PATIENCE = 7` consecutive epochs. The model checkpoint corresponding to the best validation AUC epoch was saved. The best model was obtained after no more than 11 epochs.

Evaluation

Evaluation metrics

To evaluate the success of our model and implementation, we primarily used AUC — the *Area Under the Receiver Operating Characteristic Curve* — a widely used metric for assessing the performance of classification models. The Receiver Operating Characteristic (ROC) curve plots the true positive rate (sensitivity) on the Y-axis against the false positive rate on the X-axis. Each point on the curve represents a different threshold for converting predicted probabilities into binary decisions. The curve is bounded between 0 and 1 on both axes, and the area under this curve ranges from 0.0 to 1.0, with 1.0 indicating a perfect classifier and 0.5 indicating a model that performs no better than random guessing. The ROC curve provides a threshold-independent view of how well the model separates positive from negative cases.

We also tracked recall at a 0.5 threshold for each class. Recall is calculated as $(\text{true positives}) / (\text{true positives} + \text{false negatives})$.

A recall of 1.0 means the model identified all true positives with no misses. At a fixed threshold of 0.5, any predicted probability equal to or greater than 0.5 is counted as a positive. In a medical context, recall is particularly important because missing a disease (false negative) could have serious consequences. Therefore, depending on the criticality of a finding, we might adjust the threshold to increase recall, accepting more false positives in exchange for fewer false negatives. Importantly, recall does not change how the model works internally, but rather how its outputs are interpreted.

Both AUC and recall were calculated per class (i.e., per disease label), then averaged across all 14 labels to get an overall performance view.

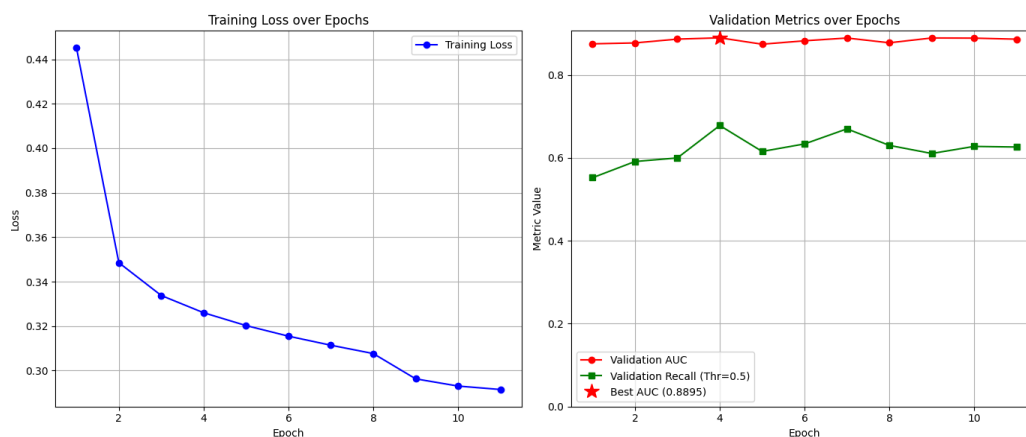


Figure 2: Plot of the highest achieved AUC for the ResNet50 backbone

```

--- Epoch 4/50 ---
Current LR: 0.000050
Training: 100%|██████████| 3491/3491 [16:15<00:00, 3.58it/s]
Validating: 100%|██████████| 4/4 [00:00<00:00, 4.89it/s]

Validation Metrics per Class:
No Finding : AUC = 0.9625, Recall@0.5 = 0.9737
Enlarged Cardiomediastinum: AUC = 0.8457, Recall@0.5 = 0.7431
Cardiomegaly : AUC = 0.8059, Recall@0.5 = 0.7206
Lung Opacity : AUC = 0.9227, Recall@0.5 = 0.8254
Lung Lesion : AUC = 0.7296, Recall@0.5 = 0.0000
Edema : AUC = 0.9090, Recall@0.5 = 0.8000
Consolidation : AUC = 0.9272, Recall@0.5 = 0.9394
Pneumonia : AUC = 0.8861, Recall@0.5 = 0.5000
Atelectasis : AUC = 0.9030, Recall@0.5 = 0.8375
Pneumothorax : AUC = 0.8313, Recall@0.5 = 0.5000
Pleural Effusion : AUC = 0.9275, Recall@0.5 = 0.7463
Pleural Other : AUC = 0.9700, Recall@0.5 = 1.0000
Skipping AUC for Fracture: Only one class present in validation ground truth.
Fracture : AUC = nan, Recall@0.5 = 0.0000
Support Devices : AUC = 0.9436, Recall@0.5 = 0.9065

Overall Mean AUC: 0.8895, Mean Recall@0.5: 0.6780

Epoch 4 Summary:
Train Loss: 0.3260
Val AUC: 0.8895 (Best: 0.8864)
Val Recall: 0.6780
Validation AUC improved (0.8864 --> 0.8895). Saving checkpoint...
Checkpoint saved to /content/drive/MyDrive/dat255/training/best_checkpoint_ResNet50.pth

```

Figure 3: Validation metrics for the best ResNet50 training epoch

Evaluation baseline

The baseline for performance comparison was effectively random guessing. A model that makes random predictions for binary classification will, on average, yield an AUC of 0.5. Thus, any AUC significantly above 0.5 demonstrates that the model is learning useful patterns. For example, our model achieved AUC scores well above 0.9 for certain conditions such as Support Devices and No Finding early in the training process, indicating strong predictive performance for these categories. In contrast, certain conditions such as *Lung Lesion* and *Fracture* proved more difficult to validate due to the lack of such cases in the validation set. The reduced performance can be attributed to a combination of factors, including the loss of fine-grained visual details due to the downsampled dataset and the limited representation of these conditions in the validation set, which constrains the model's ability to learn and generalize effectively.

Interpretation Techniques

To gain insight into what the model focuses on when making predictions, we implemented Grad-CAM (Gradient-weighted Class Activation Mapping) [16]. This technique highlights the regions of the input image that contribute most strongly to a given class prediction, helping us verify whether the model is attending to medically relevant areas.

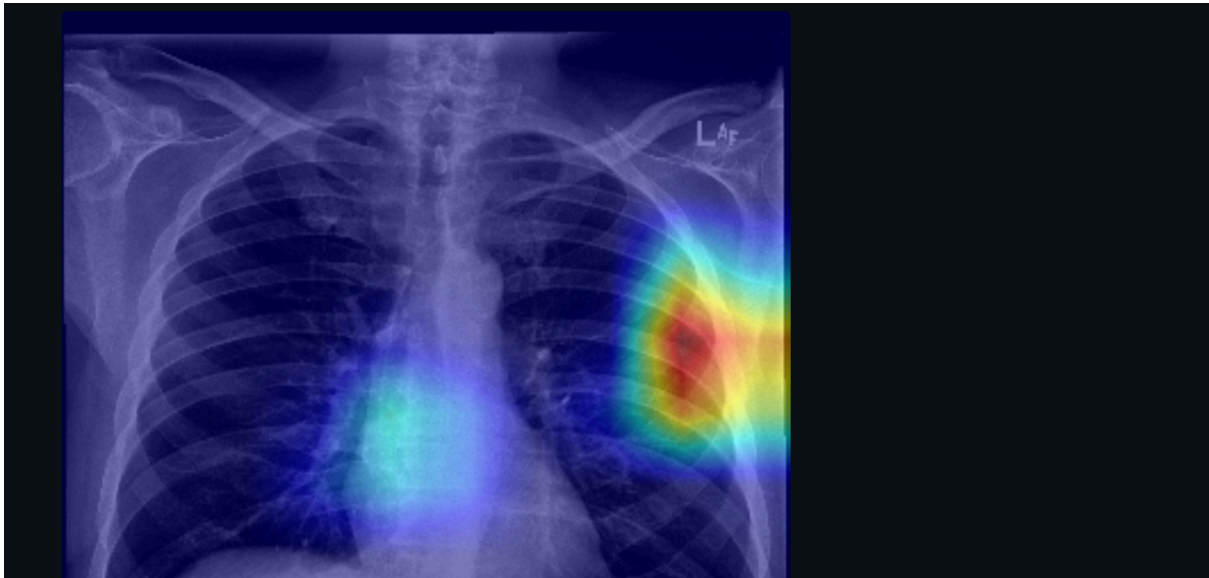


Figure 4: Example of Grad-CAM. This is highlighting where our model is observing Support Devices

Utilization of Model Architecture

Our best result was achieved using the ResNet50 architecture, with a slight boost to performance when employing additional data augmentations, the AdamW optimizer, and weight decay. The projects we are competing against are very competent research groups with far more resources. The resource limitations were the impetus to opt for a compressed data set, which limits how deep our models can be without risking overfitting. Given all the combinations of augmentations, optimizations and hyperparameters attempted, we feel that there was a limited amount of untapped potential to gain with this particular model architecture.

Inference time for all predictions on the deployed model is below one second per image with CPU inference on HuggingFace's free tier, while model attention visualization takes roughly three seconds per condition rendered.

Deployment

Deploying the model online is appropriate, as it enables users to upload chest X-ray images and receive predicted diagnosis probabilities. For this purpose, we found the generous free tier offered by HuggingFace.co to be a natural choice. Hugging Face supports automatic deployment from its integrated Git repository and manages all required dependencies seamlessly. Inference time on the CPU runtime is sufficiently fast to ensure a responsive user experience. The user interface was built using the Streamlit library, which provides a quick and straightforward way to connect the model with an interactive frontend.

There are several opportunities for improving the deployment. Functionality could be expanded by allowing users to select between different model architectures. The user experience could also be enhanced by supporting the upload and analysis of multiple images at once. Additionally, more advanced visualization techniques could be integrated, ideally accompanied by explanations and contextual information to help interpret the model's outputs. Finally, Grad-CAM visualizations could be adapted to allow users to examine one observation at a time, rather than displaying results for every label exceeding a threshold.

In a potential production environment, logging mechanisms should be implemented to monitor errors and resource usage. Ongoing maintenance would involve managing Python dependencies, including replacing deprecated libraries as needed. The model itself could benefit from retraining with new data or the integration of more advanced modeling techniques. Lastly, sustaining such a system would require ensuring continued hosting availability and scaling up infrastructure as demand increases.

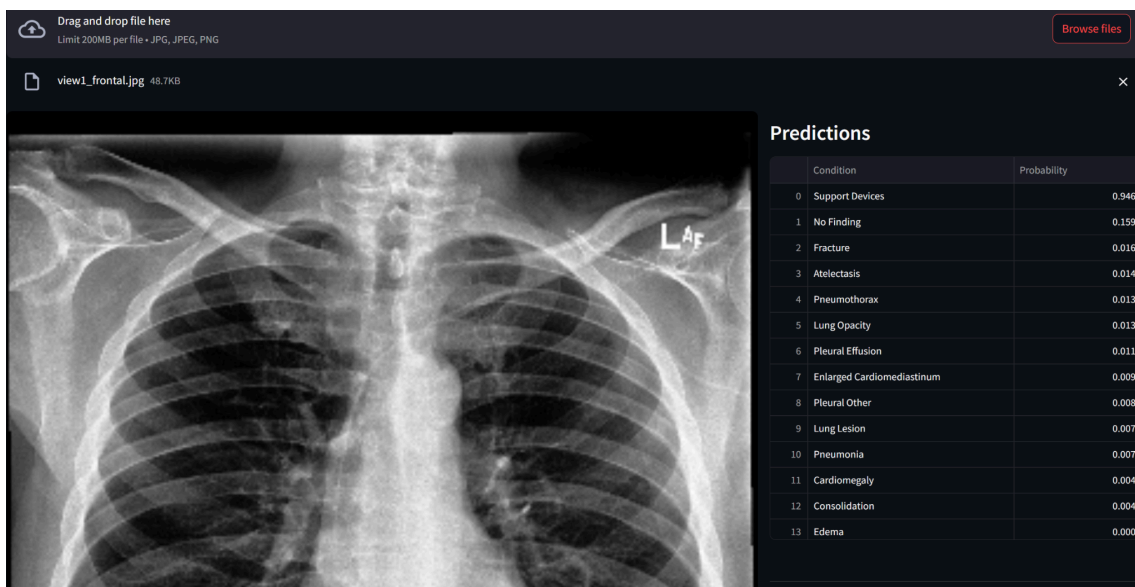


Figure 5: User interface showing the predicted observations. This is the same patient used to show Grad-CAM in Figure 4

Conclusion

The CheXpert dataset by the Stanford ML group is well-researched and has been the focus of competitive machine learning efforts. Leading research teams, equipped with considerable computational resources and domain expertise, have achieved very strong results. This illustrates both the potential and the challenge of working with this dataset, and reinforces that deep learning is a highly appropriate approach to the problem.

Our project aimed to reproduce a competitive solution under more modest conditions. According to the CheXpert leaderboard, the current top-performing model is *DeepAUC-v1*, which achieves a mean AUC of 0.9305 [5]. With this as a reference point, we consider our best model — a ResNet50 model achieving a mean AUC of 0.890 — to be a solid result, especially considering our limited resources and the use of a compressed dataset.

Using AUC as our primary evaluation metric was a natural decision. AUC is the standard benchmark for this problem and reflects the model's ability to correctly rank positive and negative cases across different threshold settings. We also monitored recall at a 0.5 threshold to evaluate sensitivity, which is particularly important in a medical context where missing a positive case could have serious consequences. Our recall was 0.678, indicating a relatively high number of false negatives compared to true positives. However, thanks to the integration of Grad-CAM and the option for users to adjust the classification threshold in the deployed interface, we believe this limitation is mitigated in practice.

We experimented with different architectures, tried alternative optimizers, and implemented regularization strategies including weight decay and augmentation. However, none of these approaches consistently outperformed our basic ResNet50 configuration. We believe this model struck the best balance based on our downsampled training data.

This project set out to develop a competitive model while operating within realistic constraints, and we believe we have successfully achieved that goal. Based on the AUC scores obtained on our validation set, our model would have ranked among the top 150 entries in the CheXpert competition. However, this ranking is only an approximation, as we did not evaluate our model on the official test set used for the competition leaderboard.

Notably, our model achieved a recall of 0 for two observations, likely due to limitations in the validation data, which raises the possibility that it could have performed better on the official test set. At the same time, we acknowledge that some of our other observations might have scored lower on the official test set than they did on our validation set.

Overall, we are satisfied with the outcome of this project. The results demonstrate that it is possible to build a reasonably competitive model for chest X-ray interpretation while minimizing costs and operating with limited computational resources compared to the top-performing entries in the CheXpert competition.

References

- [1] J. Irvin et al., "CheXpert: A Large Chest Radiograph Dataset with Uncertainty Labels and Expert Comparison", *arXiv preprint arXiv:1901.07031*, Jan. 2019. [Online]. Available: <https://arxiv.org/pdf/1901.07031>. [Accessed: Apr. 22, 2025].
- [2] Stanford Machine Learning Group, "CheXpert: A Large-Scale Chest X-ray Dataset And Competition", *Stanford ML Group*, [Online]. Available: <https://stanfordmlgroup.github.io/competitions/chexpert/>. [Accessed: Apr. 22, 2025]
- [3] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022, ch. 4
- [4] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022, ch. 14
- [5] Z. Yuan, Y. Yan, M. Sonka, and T. Yang, "Large-scale robust deep AUC maximization: A new surrogate loss and empirical studies on medical image classification," *arXiv preprint arXiv:2012.03173*, Dec. 2020. [Online]. Available: <https://arxiv.org/pdf/2012.03173>. [Accessed: Apr. 22, 2025]
- [6] H. H. Pham, T. T. Le, D. Q. Tran, D. T. Ngo, and H. Q. Nguyen, "Interpreting chest X-rays via CNNs that exploit hierarchical disease dependencies and uncertainty labels," *arXiv preprint arXiv:1911.06475*, Jun. 2020. [Online]. Available: <https://arxiv.org/pdf/1911.06475>. [Accessed: Apr. 22, 2025]
- [7] Ashery, *CheXpert-v1.0-small*, Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/ashery/chexpert>. [Accessed: Apr. 23, 2025]
- [8] National Institutes of Health, *NIH Chest X-ray Dataset*, Kaggle, 2017. [Online]. Available: <https://www.kaggle.com/datasets/nih-chest-xrays/data>. [Accessed: Apr. 23, 2025].
- [9] Johnson, A., Pollard, T., Mark, R., Berkowitz, S., & Horng, S. (2024). MIMIC-CXR Database (version 2.1.0). PhysioNet. <https://doi.org/10.13026/4jqj-jw95>.
- [10] A. E. W. Johnson, T. J. Pollard, S. J. Berkowitz, N. R. Greenbaum, M. P. Lungren, C.-y. Deng, R. G. Mark, and S. Horng, "MIMIC-CXR, a de-identified publicly available database of chest radiographs with free-text reports," *Sci. Data*, vol. 6, no. 317, 2019. [Online]. Available: <https://www.nature.com/articles/s41597-019-0322-0>. [Accessed: Apr. 23, 2025]
- [11] F. Mehrparvar, *Lung Disease*, Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/fatemehmehrparvar/lung-disease>. [Accessed: Apr. 23, 2025]

- [12] O. M. Dalvi, *Lungs Disease Dataset (4 types)*, Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/datasets/omkarmanohardalvi/lungs-disease-dataset-4-types/data>. [Accessed: Apr. 23, 2025]
- [13] S. Jain, A. Smit, S. Q. H. Truong, C. D. Nguyen, M.-T. Huynh, M. Jain, V. A. Young, A. Y. Ng, M. P. Lungren, and P. Rajpurkar, "VisualCheXbert: Addressing the discrepancy between radiology report labels and image labels," *arXiv preprint arXiv:2102.11467*, Mar. 2021. [Online]. Available: <https://arxiv.org/pdf/2102.11467>. [Accessed: Apr. 24, 2025].
- [14] Kingma, D. P. and Ba, J., "Adam: A Method for Stochastic Optimization", Art. no. arXiv:1412.6980, 2014. doi:10.48550/arXiv.1412.6980.
- [15] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022, ch. 11
- [16] J. Gildenblat, *grad-cam*, PyPI, 2025. [Online]. Available: <https://pypi.org/project/grad-cam/>. [Accessed: Apr. 24, 2025].